

How We Mapped Every EEG Dataset to 22 Channels

A Student Guide to the DiffEEG Data Pipeline

EEGdiff_V2 project

September 24, 2026

1 The problem in one sentence

Our model was born speaking **one exact “language”**: every EEG clip it reads must be **22 channels $\times$ 1280 numbers, at 256 Hz (5 seconds)**, with the 22 electrodes always in the **same order**. Every public dataset speaks a *different* language — different electrode count, names, order, and sampling rate. *Processing* means translating each dataset into the model’s one language.

2 The 22-channel “frame” (the key idea)

Picture a **muffin tin with 22 labeled cups**, in a fixed order:

FP1 FP2 F3 F4 C3 C4 P3 P4 O1 O2 F7 F8 T3 T4 T5 T6 A1 A2 FZ CZ PZ ROC

The model learned “cup #13 is always T3.” For any new dataset we must drop each electrode into the **correct cup**. If an electrode is missing, we leave that cup **empty (all zeros)**. If we put an electrode in the *wrong* cup, there is **no error message** — the model just silently produces bad results. That is why the *order* matters as much as the names.

3 The universal 4-step recipe

Almost every dataset goes through the same four steps (`dataset_tools/prepare_external_eeg.py`).

Step A — clean the electrode name so that “EEG T7-REF” and “t7” both become “T7”:

```
def canonical_channel(name):
    cleaned = re.sub(r"[^A-Za-z0-9]", "", name.upper()) # drop spaces/dashes
    cleaned = cleaned.replace("EEG", "") # drop the "EEG" prefix
    return ALIASES.get(cleaned, cleaned) # apply the alias table
```

Step B — the alias table because different labs use different names for the *same* spot on the head:

```
TARGET_CHANNELS = ["FP1", "FP2", "F3", "F4", "C3", "C4", "P3", "P4", "O1", "O2", "F7",
                    "F8", "T3", "T4", "T5", "T6", "A1", "A2", "FZ", "CZ", "PZ", "ROC"]
ALIASES = {
    "T7": "T3", "T8": "T4", "P7": "T5", "P8": "T6", # modern name -> old name
    "TP9": "A1", "TP10": "A2", "FPZ": "FP1",
    "FPZCZ": "FZ", "PZ0Z": "PZ", # sleep-EDF's bipolar pairs
}
```

Step C — place into the 22 cups, zero-fill the rest. This is the actual “mapping”:

```
def project_channels(data, names):
    out = np.zeros((22, data.shape[1]), dtype=np.float32) # start all-zero
    present = {}
    for i, name in enumerate(names):
        c = canonical_channel(name)
        if c in TARGET_CHANNELS and c not in present:
            present[c] = i # this electrode is cup c
    for j, target in enumerate(TARGET_CHANNELS):
        if target in present:
            out[j] = data[present[target]] # copy signal into its cup
    return out, sorted(set(TARGET_CHANNELS) - set(present)) # + list of empty cups
```

Step D — resample to 256 Hz and cut into 5-second windows:

```
def resample_full(data, sfreq, target_sfreq): # e.g. 512 Hz -> 256 Hz
    up, down = int(target_sfreq), int(round(sfreq))
    return resample_poly(data, up, down, axis=-1)
# then slide a 1280-sample window across the recording
```

That is the whole trick. Below is what was **special** for each dataset.

4 The processing script for every dataset

TUSZ & TUAB — the “home” datasets (exact-name match)

These use the same clinical montage the model was trained on, so the names already match (EEG FP1-REF → FP1) — no aliases, just extract the 22 by name, resample 250 → 256 Hz, and window. (Processed by a collaborator; the recipe is recorded in TUAB/processed_tuab/tuab_processing_report.json.) **Result: 22/22 cups filled.**

Bonn — only one electrode exists

Bonn is a text file with a **single** signal (no electrode name), 173.61 Hz. We put it in cup #1 (FP1) and leave the other 21 empty.

```
def prepare_bonn(raw_root, writer, args):
    for txt in txt_files:
        label = "seizure" if stem.startswith("S") else "normal"
        data = np.loadtxt(txt)[None, :] # one channel
        data = resample_to_target(data, 173.61, 256, ...) # -> 256 Hz
        framed = np.zeros((22, data.shape[1])) # 22 empty cups
        framed[0] = data[0] # put signal in FP1
        writer.add(split_name(stem), label, framed)
```

Result: 1/22 cups filled.

EEGMMIDB (motor imagery) — 64 electrodes, keep 19

A 64-electrode cap, 160 Hz. The alias table maps 19 of them onto our frame; the 3 clinical channels it lacks (A1, A2, ROC) stay empty. Labels come from the EDF *annotations* (T1 = left fist, T2 = right fist).

```
def prepare_eegmmidb(raw_root, writer, args):
    for edf in ...:
        spans = [(onset, onset+dur, t1_or_t2_label) for each annotation]
        window_edf(edf, writer, "eegmmidb", subject, spans, "rest", args)
        # window_edf does: project_channels -> resample_full -> slide window,
        # tagging each window by which annotation its center falls in
```

Result: 19/22 cups filled.

TUEV (event types) — same montage, but tiny event labels

Channel mapping is the same `project_channels`. The special part is *windowing*. TUEV has been processed **twice**, with two different windowing methods, and only the second one matches how published papers use it.

The problem: it is the *labels* that are 1 second, not the input

The EEG **recordings** are long and normal (e.g. 10 minutes). What is short is the **labels**: TUEV annotates the data in tiny **1-second, per-channel** chunks. Each `.rec` line is `channel, start_sec, stop_sec, label` — e.g. 14, 1.4, 2.4, 5 means “on channel 14, seconds 1.4–2.4, label 5 (artifact).” So “1 second” is the *granularity of the annotations*, not the size of the input we feed the model (which is still 5 s). Every annotation is exactly 1.0 s long, and the channel number (0–21) refers to the 22-channel TCP bipolar montage.

Why that breaks the naive approach. Our model eats **5-second** windows.

- **TUSZ / TUAB** label *continuously* — a whole recording (or a long span) is “seizure”/“normal.” So you chop it into a fixed grid (0–5 s, 5–10 s, ...) and every window inherits a clear label. Easy.
- **TUEV** labels are *sparse 1-second events* scattered through a 10-minute recording. Tile that same fixed 5-second grid and ask “is there an event at this window’s center?” — the little 1-second markers almost never line up with the grid cells, so you throw away ~98% of windows.

Both methods below fix this by cutting windows *around the events* instead of on a fixed grid. They differ in **how many windows each event produces**.

First attempt (used in V1-era benchmarking and V2; now retired). Script: `dataset_tools/prepare_tuev.py`. It read the `.rec` rows but **ignored the channel column**, then ran `merge_events` to join back-to-back rows with the same label into one long span, cut **one 5-second window per span**, and dropped windows that started at the same time.

```
events = [(start, stop, label) for _ch, start, stop, label in rows] # channel thrown away
spans = merge_events(events) # [0.4, 1.4), [1.4, 2.4), ... -> [0.4, 6.4)
for s0, s1, lab in spans:
    start = center a 5 s window on the span # ONE window per span
```

Result: 2,665 windows in total (V2’s fine-tuning used 2,621 of them). Output folders: `processed_TUEV/` and `processed_TUEV_4class/`. **Do not use these for new experiments:** the numbers they produce cannot be compared with any published TUEV result.

Current method: the BIOT/LaBraM protocol. Script: `dataset_tools/prepare_tuev_events.py`. This is how BIOT defined TUEV, and LaBraM, CBraMod and EEGDM all copied it:

one sample per .rec row. Each row’s 1-second event gets a 5-second window, 2s before and 2s after, and the row’s channel is kept as the “offending channel.”

```
for ch, s0, s1, label in parse_rec(rec_path): # every row, no merging
    start = s0 - 2.0 # 2 s before the 1 s event
    window = signal[:, start : start + 5.0] # all 22 channels, 5 s
    # clamp the window inside the recording if the event is near an edge
    save(window, label, offending_channel=ch, clamped=...)
```

Worked example: why the counts differ by 43×

An artifact that lasts **6 seconds** and is marked on **3 channels** is written as $6 \times 3 = 18$ rows in the .rec file.

- **First attempt:** the 18 rows are merged into one span → about **1 window**.
- **Current method:** each row is its own sample → **18 samples**, each labeled with the channel it came from.

The question each sample asks also changes, from “what is going on in this 5-s stretch?” to “what is this 1-second event on this channel, given the 5s around it?” The second question is the one published TUEV results answer.

	First attempt	Current (BIOT protocol)
Samples (train + test)	2,665	113,353 (83,932 + 29,421)
Spike-and-slow-wave (spsw)	~78 total	645 train / 567 test
Which channel has the event	discarded	kept
Comparable with published TUEV	no	yes

Class counts, current method (train / test): spsw 645 / 567, gped 11,254 / 4,677, pled 6,184 / 1,998, eyem 1,070 / 329, artf 11,053 / 2,204, bckg 53,726 / 19,646.

Where it is: Dataset/TUEV_v2.0.1/processed_TUEV_events/, one folder per split and class (train-spsw/, test-gped/, ...). Each <class>_batch_NNNN.npy holds [N, 22, 1280] windows and the matching _meta.npz holds, per sample, offending_channel, session, event_start and clamped.

Things to know before using it:

- **Volts, not normalized.** Values are around 10^{-5} . Normalize with TUEV’s own statistics (golden rule 2 below), or the model sees a flat signal.
- **Keep the train/test split.** It is TUEV’s own and patient-disjoint.
- **Samples overlap heavily, on purpose.** An event marked on several channels produces the same 5s window once per channel: 113,353 samples come from only 13,020 distinct windows. That is correct for classification but **wrong for pre-training**, where the model would see some windows 22 times.
- **Two deliberate differences from BIOT.** We keep our 22 monopolar channels at 256 Hz (BIOT converts to 16 bipolar channels at 200 Hz). And for events within 2s of a recording edge (6,959 samples, flagged clamped=1), we shift the window to fit inside the recording; BIOT instead wraps around and pastes in signal from the other end of the recording.

Also: TUEV as pre-training data for V3 (no labels). Script: diff-eeg-V3/data/build_pool.py. For pre-training we do not want event windows at all, so the **raw continuous recordings**

are cut into non-overlapping **30-second windows at 200 Hz on 21 channels** (ROC dropped, see below). Result: **17,678 windows = 147 hours**, in Dataset/V3_pool_30s_200hz/tuev/. This folder is for pre-training only; it has no labels.

Result: 21/22 cups filled in every sample; ROC is filled in only about 60% of samples, because many TUEV recordings do not include that eye channel. (This is also why V3 drops ROC and uses 21 channels.)

Sleep-EDFx — only 2 bipolar channels

The hardest case: only **2 EEG channels**, both *bipolar pairs* (Fpz-Cz, Pz-Oz) that match nothing in a normal montage. The two aliases FPZCZ→FZ and PZOZ→PZ rescue it — without them, *all 22 cups end up empty* (which happened once and produced garbage until we caught it). Sleep-stage labels come from a separate Hypnogram file, paired by shared filename prefix.

```
def prepare_sleep_edfx(raw_root, writer, args):
    for psg in psgs:
        hyp = next(psg.parent.glob(psg.name[:7] + "*Hypnogram.edf"), None) # pair
        spans = [(onset, onset+dur, sleep_stage) for each annotation]
        window_edf(psg, writer, "sleep_edfx", subject, spans, "sleep", args)
        # project_channels maps Fpz-Cz -> FZ, Pz-Oz -> PZ, the rest stay zero
```

Result: 2/22 cups filled.

5 Siena — the special case (processed by Yehya)

Siena is different from all the others: it was **not** run through our pipeline. Our collaborator **Yehya** had already processed it, and his output *already matched our model's spec*: 22 channels, in the right order, at 256 Hz, in 1280-sample (5s) windows. So there was **no channel mapping and no resampling to do**. Our script `dataset_tools/convert_siena_npz.py` only re-packages his .npz files into our folder layout, with a clean *patient-wise* train/test split.

```
for patient, recs in by_patient.items():
    split = "test" if patient in test_patients else "train" # no patient in both
    for f in recs:
        d = np.load(f); X, Y = d["signals"], d["labels"]
        for x, y in zip(X, Y):
            label = "seizure" if y == 1 else "normal"
            buffers[(split, label)].append(x) # x is already 22x1280
```

Did we change the volts? (short answer: no)

Yehya’s Siena data is stored in **volts**, so the numbers are tiny (standard deviation $\approx 2 \times 10^{-4}$). Our model was trained on a different scale (standard deviation ≈ 230 , about a **million times bigger**). **We did *not* rescale or convert the volts** — we kept Yehya’s raw values exactly as they were.

The fix happens at **normalization time**. Before the model sees a signal we always z-score it: $x_{\text{norm}} = (x - \text{mean})/\text{std}$. The trick is *which* mean and std we use:

- **What we do (correct):** use **Siena’s own** mean/std, which are also in volts (std $\approx 2 \times 10^{-4}$). Dividing volts by a volts-scale std cancels the unit and gives a clean signal with std ≈ 1 — exactly what the model expects.
- **What would break it (the bug we avoided):** using *THUSZ’s* microvolt-scale std (≈ 230) on Siena’s volt data would give $2 \times 10^{-4}/230 \approx 10^{-6}$ — an essentially flat, dead signal, and the model would predict garbage.

In one line: we did not touch the volts; we just normalized Siena with *its own* statistics, which makes the physical unit irrelevant. When running any eval/finetune script on Siena, point `--norm-stats-dir` at `Siena/processed_siena/normalization` (Yehya’s stats), *never* at the THUSZ normalization folder.

Result: 19/22 cups filled (A1, A2, ROC are absent in Siena’s montage).

6 Summary

Dataset	Special handling	Cups filled	Script
TUSZ / TUAB	exact-name match	22/22	collaborator’s (see report)
Bonn	single trace $\rightarrow$ FP1	1/22	<code>prepare_bonn</code>
EEGMMIDB	alias 19; labels from annotations	19/22	<code>prepare_eegmmidb</code>
Siena	re-package; keep volts, use own stats	19/22	<code>convert_siena_npz.py</code>
TUEV	one window per <code>.rec</code> row	21–22/22	<code>prepare_tuev_events.py</code>
TUEV, V3 pre-train	30s continuous, 21 ch	21/21	<code>build_pool.py</code>
Sleep-EDFx	bipolar $\rightarrow$ midline aliases	2/22	<code>prepare_sleep_edfx</code>

The three golden rules to remember:

1. **Order matters as much as names.** A right-set-wrong-order montage fails silently.
2. **Normalize with each dataset’s own statistics.** That is what makes Siena’s volts (or any scale) work without touching the raw numbers.
3. **Split so the test set is genuinely unseen — and so it matches whatever you compare against.** Patient-wise for clinical datasets (Siena, TUSZ); subject-wise for EEGMMIDB. A random window split leaks the same person — and often the same recording — into both sides and inflates the result.